



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2283 - DISEÑO Y ANÁLISIS DE ALGORITMOS

Ayudantía 2 - Caso Promedio y Cotas Inferiores

Agosto de 2025

Caetano Borges, Manuel Villablanca

1. Algoritmo del mínimo: cotas inferiores y caso promedio

Algorithm 1 Min

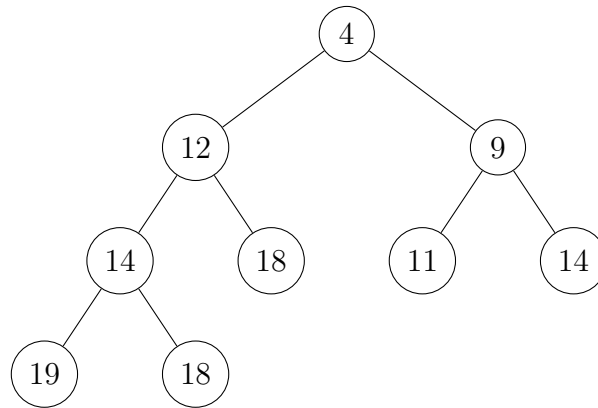
```
1: Input: Un arreglo  $A[1 \dots n]$ 
2: Output: El elemento mínimo de  $A[1 \dots n]$ 
3:  $min \leftarrow \infty$ 
4: for  $i = 1$  to  $n$  do
5:   if  $A[i] < min$  then
6:      $min \leftarrow A[i]$ 
7:   end if
8: end for
9: return  $min$ 
```

1. Use árboles de decisión para demostrar una cota inferior para la cantidad de comparaciones necesarias para resolver el problema de encontrar el elemento mínimo de un arreglo.
¿Es esta la mejor cota inferior para el problema?
- 2.Cuál es la complejidad, en torno a las asignaciones a la variable min , del:
 - a) Mejor caso. ¿Contradice esto a la cota inferior demostrada en el inciso anterior?
 - b) Peor caso
 - c) Caso promedio

Asuma que todos los elementos son distintos.

2. Caso Promedio Heap

Tal como se estudió en cursos anteriores (como Estructuras de Datos y Algoritmos), un MinHeap es un árbol binario casi lleno (es decir, que todos los niveles, excepto posiblemente el último, están completamente llenos, y los nodos en el último nivel están lo más a la izquierda posible), en el que se cumple que el valor almacenado en cada nodo del árbol es menor o igual que el valor almacenado en sus hijos (si es que estos existen). Así, un Min-Heap de n nodos consiste de $\lfloor \lg_2 n \rfloor + 1$ niveles. Un ejemplo de Min-Heap de 10 elementos enteros es el siguiente:



Además de las operaciones típicas de un Min-Heap (insertar, borrar mínimo, obtener mínimo), una operación de interés para muchas aplicaciones es **DecrecerValor** (x, v), para un nodo x y un valor entero v . Si el valor almacenado en el nodo x es $\text{valor}(x)$, la operación modifica ese valor para almacenar $\text{valor}(x) - v$ (es decir, decrementa su valor en v). La operación puede necesitar reestructurar el heap para que se cumpla la condición de orden entre los nodos, posiblemente haciendo “flotar” a x hacia arriba en el árbol, hasta encontrar la posición en donde el orden requerido se cumpla. El algoritmo es el siguiente:

Algorithm 2 DecrecerValor

- 1: **Input:** Un nodo x y un valor v
 - 2: **Efecto:** Almacenar $\text{valor}(x) - v$ en el nodo x .
 - 3: $y \leftarrow x$
 - 4: **while** $y \neq \text{raíz}$ and $\text{valor}(y) < \text{valor}(\text{padre}(y))$ **do**
 - 5: intercambiar valores entre y y $\text{padre}(y)$
 - 6: $y \leftarrow \text{padre}(y)$
 - 7: **end while**
-

Demuestre formalmente que una ejecución de dicho algoritmo realiza $\sim \lfloor \lg_2 n \rfloor$ comparaciones en promedio del tipo “ $\text{valor}(y) < \text{valor}(\text{padre}(y))$ ” (en la línea 3). Para simplificar su análisis, puede asumir que $n = 2^k - 1$, para algún $k \geq 1$. Haga las suposiciones que considere necesarias para que su análisis sea válido.